



GLAXIT

INTERNSHIP PROGRAM

Advanced Cyber Security

Task 2 Report

Asset Discovery & Asset Inventory Management Report

Submitted by: Hasnain Ali

Program: Advanced Cyber Security

Submitted to: Sir Saifullah

Date: 9th August 2026

Asset Discovery & Asset Inventory Management Report

Vulnerability Assessment & Patch Management — Phase 1: Asset Discovery

1. Objective & Scope

Asset discovery is the foundation of vulnerability assessment and patch management: an organization cannot assess risk on, or patch, assets it has not identified. This exercise performs a practical asset discovery and inventory pass on one authorized system — a personally-owned Kali Linux virtual machine — covering hardware, operating system, network interfaces, user accounts, installed applications/services, and open/listening ports.

All commands used were non-destructive, read-only enumeration commands run from a standard (non-root) shell. No configuration changes were made to the system at any point during this exercise, and the scope was limited to this single authorized machine.

2. Methodology / Tools Used

Native Linux command-line utilities were used to enumerate each asset category, as summarized below. Full command output is reproduced in §4 (Evidence).

Command	Purpose
<code>cat /etc/passwd grep -E "bash zsh"</code>	Enumerate local user accounts with interactive login shells
<code>ip -br addr show</code>	Enumerate network interfaces and assigned IP addresses
<code>cat /etc/os-release</code>	Identify OS distribution, version, and codename
<code>uname -mrs</code>	Identify kernel version and CPU architecture
<code>lscpu</code>	Inventory CPU hardware and check hardware vulnerability / mitigation status
<code>ss -tulpn</code>	Enumerate listening TCP/UDP ports and bound processes
<code>free -h</code>	Inventory RAM / swap capacity and current usage

3. Asset Inventory

Fourteen assets were identified across six categories. The Importance column reflects each asset's relative significance in a vulnerability-assessment / patch-management context (see shading key: Critical / High / Medium / Low).

Asset ID	Asset Name	Category	Description	Version / Details	Owner / User	Importance	Current Status
HW-01	CPU (Host Processor)	Hardware	Processor allocated to the VM via KVM virtualization	Intel Core i5-8365U @ 1.60GHz; x86_64; 2 vCPUs (1 socket × 2 cores × 1 thread)	HasnainAli	High	Active — 2 CPU vulnerability classes flagged “Vulnerable”, see §6
HW-02	System Memory (RAM/Swap)	Hardware	Physical memory and swap available to the VM	1.9 GiB RAM (875 MiB used, 1.1 GiB available); 953 MiB swap, ~unused	HasnainAli	Medium	Active — healthy headroom, no memory pressure
OS-01	Operating System	Operating System	Base OS distribution running on the host	Kali GNU/Linux Rolling, VERSION_ID 2026.3 (kali-rolling)	HasnainAli	Critical	Active — rolling release; confirm update cadence
OS-02	Kernel	Operating System	Linux kernel powering the OS	Linux 7.0.12+kali-amd64, x86_64	HasnainAli	Critical	Active — verify against current Kali kernel / CVE feed
NET-01	Loopback Interface (lo)	Network Interface	Internal-only loopback interface	127.0.0.1/8, ::1/128	System	Low	Active — local-only, minimal exposure
NET-02	Ethernet Interface (eth0)	Network Interface	Primary interface; only externally-reachable NIC	IPv4 10.0.2.15/24 (NAT); IPv6 fd17:625c:f037:2:338a:d6bd:1170:dab9/64; link-local fe80::2387:5919:399:26af/64	HasnainAli	High	Active — primary network attack surface
NET-03	Docker Bridge (docker0)	Network Interface	Virtual bridge created by Docker Engine for container networking	172.17.0.1/16 — interface state: DOWN	HasnainAli	Medium	Present but inactive — confirm if still required
USR-01	root	User Account	Built-in superuser account	UID 0 / GID 0, shell /usr/bin/zsh, home /root	root	Critical	Active — verify SSH / direct root-login policy
USR-02	postgres	User Account (Service)	Service account owning the PostgreSQL engine	UID 115 / GID 115, shell /bin/bash, home /var/lib/postgresql	System (service)	High	Active — confirm PostgreSQL version / running status
USR-03	kali	User Account	Default account created by the Kali Linux installer	UID 1000 / GID 1000, shell /usr/bin/zsh, home /home/kali	Kali (default)	Medium	Active — confirm still used; disable if not needed
USR-04	HasnainAli	User Account	Primary interactive / administrative account	UID 1001 / GID 1001, shell /bin/bash, home /home/HasnainAli	HasnainAli	Critical	Active — primary account used for this assessment

Asset ID	Asset Name	Category	Description	Version / Details	Owner / User	Importance	Current Status
APP-01	PostgreSQL (inferred)	Installed Application	DB engine; presence inferred from dedicated service account	Version not captured in this pass	HasnainAli	High	Inferred only — confirm with <code>dpkg -l / psql --version</code>
APP-02	Docker Engine (inferred)	Installed Application	Container platform; presence inferred from docker0 bridge	Version not captured in this pass	HasnainAli	Medium	Inferred only, bridge down — confirm with <code>docker version</code>
SVC-01	Unidentified Listener (:43619)	Open Port / Service	Process bound to a loopback-only TCP port; identity unresolved	TCP 127.0.0.1:43619, state LISTEN, Recv-Q 0 / Send-Q 4096	Unknown	High	Needs investigation — rerun <code>sudo ss -tulpn / lsof</code>

4. Evidence — Command Output

The screenshots below were captured directly from the terminal during this exercise, in the order the commands were run.

Evidence 1 — `cat /etc/passwd | grep -E "bash|zsh"` — user accounts (USR-01-04)

```
(HasnainAli@kali)~$ cat /etc/passwd | grep -E "bash|zsh"
root:x:0:0:root:/root:/usr/bin/su
postgres:x:115:115:PostgreSQL administrator:/var/lib/postgresql:/bin/bash
kali:x:1000:1000::/home/kali:/usr/bin/zsh
HasnainAli:x:1001:1001:Hasnain Ali,1,1,1,1:/home/HasnainAli:/bin/bash
```

Evidence 2 — `ip -br addr show` — network interfaces (NET-01-03)

```
(HasnainAli@kali)~$ ip -br addr show
lo        UNKNOWN    127.0.0.1/8 ::1/128
eth0      UNKNOWN    10.0.2.15/24 fd17:625c:f037:2:338a:d6bd:1170:dab9/64 fe80::2387:5919:399:26af/64
docker0   DOWN       172.17.0.1/16
```

Evidence 3 — `cat /etc/os-release` — operating system (OS-01)

```
(HasnainAli@kali)~$ cat /etc/os-release
PRETTY_NAME="Kali GNU/Linux Rolling"
NAME="Kali GNU/Linux"
VERSION_ID="2026.3"
VERSION="2026.3"
VERSION_CODENAME=kali-rolling
ID=kali
ID_LIKE=debian
HOME_URL="https://www.kali.org/"
SUPPORT_URL="https://forums.kali.org/"
BUG_REPORT_URL="https://bugs.kali.org/"
ANSI_COLOR="1;31"
```

Evidence 4 — uname -mrs — *kernel (OS-02)*

```
(HasnainAli@kali)-[~]
$ uname -mrs
Linux 7.0.12+kali-amd64 x86_64
```

Evidence 5 — lscpu — *CPU hardware & vulnerability status (HW-01)*

```
(HasnainAli@kali)-[~]
$ lscpu
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Address sizes:          39 bits physical, 48 bits virtual
Byte Order:             Little Endian
CPU(s):                 2
On-line CPU(s) list:    0,1
Vendor ID:              GenuineIntel
Model name:             Intel(R) Core(TM) i5-8365U CPU @ 1.60GHz
CPU family:             6
Model:                  142
Thread(s) per core:     1
Core(s) per socket:     2
Socket(s):              1
Stepping:               12
Microcode version:      0xffffffff
BogoMIPS:               3792.01
Flags:                  fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush mmx fxsr sse sse2 ht syscall nx rdtscp lm constant_tsc rep_good nopl xtopology nonstop_tsc cpuid tsc_known_freq pni pclmulqdq s
apabilities
Virtualization features:
Hypervisor vendor:     KVM
Virtualization type:   full
Caches (sum of all):
L1d:                   64 KiB (2 instances)
L1i:                   64 KiB (2 instances)
L2:                    512 KiB (2 instances)
L3:                    12 MiB (2 instances)
NUMA:
NUMA node(s):          1
NUMA node0 CPU(s):     0,1
Vulnerabilities:
Gather data sampling:   Unknown: Dependent on hypervisor status
Ghostwrite:            Not affected
Indirect target selection: Mitigation; Aligned branch/return thunks
Itlb multihit:         KVM: Mitigation: VMX unsupported
L1tf:                  Not affected
Mds:                   Not affected
Meltdown:              Not affected
Mmio stale data:       Vulnerable: Clear CPU buffers attempted, no microcode; SMT Host state unknown
Old microcode:         Not affected
Reg file data sampling: Not affected
Retbleed:              Mitigation; Enhanced IBRS
Spec rstack overflow:  Not affected
Spec store bypass:     Vulnerable
Spectre v1:            Mitigation; usercopy/swapgs barriers and __user pointer sanitization
Spectre v2:            Mitigation; Enhanced / Automatic IBRS; PBRSE-eIBRS SW sequence; BHI SW loop, KVM SW loop
Srbds:                 Unknown: Dependent on hypervisor status
Tsa:                   Not affected
Tsx async abort:       Not affected
```

Evidence 6 — ss -tulpn — *listening ports (SVC-01)*

```
(HasnainAli@kali)-[~]
$ ss -tulpn
```

Netid	State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port	Process
tcp	LISTEN	0	4096	127.0.0.1:43619	0.0.0.0:*	

Evidence 7 — free -h — memory (HW-02)

```
(HasnainAli@kali)-[~]
$ free -h
              total        used        free      shared  buff/cache   available
Mem:           1.9Gi       875Mi       152Mi        15Mi        1.1Gi       1.1Gi
Swap:          953Mi         4.0Ki       953Mi
```

5. Top 3 Priority Assets for Vulnerability Assessment & Patch Management

1. Operating System & Kernel — OS-01, OS-02

Kali GNU/Linux Rolling 2026.3, kernel 7.0.12+kali-amd64. The OS and kernel are the foundation every other asset depends on: every application, service, and available security mitigation runs on top of them. The lscpu evidence (§4, Evidence 5) shows two hardware-vulnerability classes reported “Vulnerable” (MMIO Stale Data, Speculative Store Bypass) — whether these can be mitigated is itself a function of kernel and microcode support. Kali's rolling-release model also means the installed version drifts quickly, so patch status must be actively verified rather than assumed current.

2. Privileged & Service Accounts — USR-01 (root), USR-02 (postgres)

Compromise of the root account yields complete control of the system; compromise of the postgres service account could expose an entire database. These accounts carry the highest impact if compromised and should be first in line for access-control review — authentication method, SSH exposure, and password/sudo policy — ahead of lower-privilege accounts.

3. Network-Facing Surface — NET-02 (eth0), SVC-01 (unidentified listener)

eth0 is the system's only externally-reachable interface, making it the actual attack surface for this machine, and the unidentified process on TCP 127.0.0.1:43619 is by definition an unverified asset until it is positively identified. Under a standard risk model, anything both network-adjacent and unknown is escalated for review until it is confirmed benign.

6. Unknown, Outdated, or Suspicious Assets Requiring Further Investigation

- **Unidentified process on TCP 127.0.0.1:43619 (SVC-01):** the owning process was not visible because ss was run without elevated privileges. Re-run as `sudo ss -tulpn` or `sudo lsof -i :43619 -P -n` to resolve its identity before it can be risk-rated.
- **Two CPU vulnerability classes reported “Vulnerable”:** MMIO Stale Data (no microcode fix available in this environment) and Speculative Store Bypass. Confirm whether host-level microcode / hypervisor patches exist, since mitigation from inside the guest alone is limited.
- **CPU “Microcode version” reads 0xffffffff:** this placeholder is typical for many virtualized guests, which are not given the physical host's real microcode revision — flag for host-level verification rather than treating it as a literal missing patch.
- **docker0 bridge present but DOWN (NET-03):** Docker appears installed but unused. Confirm whether it is still required; unused software increases attack surface with no offsetting benefit.
- **Default kali account alongside HasnainAli (USR-03 / USR-04):** two potentially valid interactive login accounts exist on what appears to be a single-user machine. Confirm the kali account is still in use, or lock/disable it to reduce the login surface.

- **PostgreSQL and Docker presence are inferred, not confirmed (APP-01, APP-02):** both were identified indirectly (a service account and a bridge interface, respectively) rather than via a direct package/version query. Versions and running status should be confirmed before scoring them in a full vulnerability assessment.

7. Recommended Next Steps

- Re-run port enumeration with elevated privileges (`sudo ss -tulpn / lsof`) to close the SVC-01 identification gap.
- Capture a full installed-package inventory (`dpkg -l, apt list --installed`) to confirm APP-01 / APP-02 versions and surface any others not visible from this pass.
- Feed the confirmed OS/kernel version and the CPU vulnerability findings into a vulnerability scanner as the next phase of the assessment.
- Establish a recurring inventory refresh cadence, given the rolling-release OS and the pace at which its packages change.